МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 1

з дисципліни „Функціональне програмування"
на тему „Основи функціонального програмування у Python"
Варіант: 4 — „Кошик покупця"

Виконав
студент 2 курсу
групи ФЕП-23с
Данилюк Андрій Андрійович

Перевірив
Свелеба Сергій Андрійович

Львів – 2026

Мета роботи

Зрозуміти, що таке чиста функція і референтна прозорість; навчитися виявляти та ізолювати побічні ефекти; переписати імперативний фрагмент у функціональному стилі; застосувати typing.Callable для параметризації обчислень.

Варіант: 4 — Кошик покупця

Товари мають назву, ціну, кількість і категорію. Потрібно обчислити subtotal, категоріальну знижку, знижку за купоном і ПДВ; правила знижки та податку передати через Callable і повернути новий кошик, не змінюючи вхідні дані.

Репозиторій: https://github.com/AndriiDandr/FP, папка Lab_01.

Хід роботи

Чисте ядро (core.py). Стартовий фрагмент process_orders переписано як process_orders_pure / make_processor без глобального TAX_RATE, print і мутацій. Для варіанта реалізовано чисті функції line_total, cart_subtotal (reduce), price_item, with_qty, with_price, without_category (filter), scale_prices (map), process_cart_pure — усі повертають нові об’єкти.

Політики через typing.Callable. Фільтр позицій, категоріальна знижка, купон і ПДВ описані типами Callable (ItemFilterFn, CategoryDiscountFn, CouponFn, TaxFn). Функція вищого порядку make_cart_processor збирає з них обробник кошика; фабрики make_category_discount, make_percent_coupon, make_fixed_coupon, make_vat повертають налаштовані політики; compose дозволяє комбінувати купони.

Ізоляція побічних ефектів (app.py). Читання cart.json, системний час і print є лише в оболонці. Час передається в ядро як функція: stamp_total(total, time.time).

Тести та якість коду. 29 pytest-тестів, зокрема property-based (Hypothesis) і тести еквівалентності з імперативною версією starter_imperative.py: референтна прозорість, відсутність мутацій, правильність Callable-політик і розрахунків. mypy --strict, black і ruff — без зауважень; перевірки запускаються в GitHub Actions.

Результат роботи

```
> python app.py
=== Кошик: купон 10%, ПДВ 20% ===
Товар                Категорія         Ціна  К-сть      Сума  Знижка     Після
------------------------------------------------------------------------------
Ноутбук Lenovo       electronics   24999.00      1  24999.00      5%  23749.05
Мишка Logitech       electronics     650.00      2   1300.00      5%   1235.00
Книга "Python"       books           480.00      3   1440.00     10%   1296.00
Футболка             clothes         350.00      2    700.00     15%    595.00
Кава зернова         food            289.50      4   1158.00      0%   1158.00
------------------------------------------------------------------------------
Subtotal:                  29597.00
Категоріальна знижка:      -1563.95
```

```
Купон:                      -2803.31
ПДВ:                        +5045.95
До сплати:                  30275.69
Час розрахунку: 2026-10-16 13:29:22
```

```
> python -m pytest
29 passed in 1.25s
> python -m mypy
Success: no issues found in 10 source files
> python -m black --check .
10 files would be left unchanged.
> python -m ruff check .
All checks passed!
```

Перевірка: subtotal 29597,00 грн; категоріальна знижка 1563,95 грн; купон 10% — 2803,31 грн; ПДВ 20% — 5045,95 грн; до сплати 30275,69 грн. Товар з кількістю 0 відкинуто фільтром. Заміна функцій-політик змінює розрахунок без змін у core.py.

Висновки

У цій лабораторній роботі я розділив програму на чисте ядро та оболонку побічних ефектів. Ядро не має I/O, глобального стану і мутацій — кожна функція повертає новий об’єкт, тому результат залежить лише від аргументів. Політики розрахунку передаються через typing.Callable, а функція вищого порядку збирає з них обробник, тож правила легко замінювати. Тести, зокрема property-based, підтвердили референтну прозорість і відсутність мутацій, а mypy, black і ruff не знайшли зауважень.